Improved Simulation of Stabilizer Circuits

Ashwin

Need for better simulation

In general, a quantum state looks like

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} a_x |x\rangle$$

For simulating n qubits, 2^n complex numbers are required.

This quickly becomes unfeasible

$$2^{10} = 1024$$

$$2^{64} = 18446744073709551616$$

What's a stabilizer circuit?

- Circuits consisting of only Hadamard, CNOT and Phase P gates + 1 qubit measurement gates
- Circuits formed by first 3 are called unitary circuits.
- For a n qubit stablizer state $|\psi\rangle$
  1. $|\psi\rangle$ can be obtained from $|0\rangle^{\otimes n}$ by CNOT, Hadamard, and phase gates only.
  2. $|\psi\rangle$ can be obtained from $|0\rangle^{\otimes n}$ by CNOT, Hadamard, and phase gates and measurement gates only.

Pauli Matrices

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Also,

$$X^2 = I \quad Y^2 = I \quad Z^2 = I$$
$$XY = iZ \quad YZ = iX \quad ZX = iY$$

Reversing the multiplication introduces a negative sign

$$YX = -iZ$$

We define the group P_n of n-qubit Pauli operators to consist of all tensor products of n Pauli matrices, together with a multiplicative factor of ± 1 or $\pm i$

$$P_n = i^k P_1 P_2 P_3 ... P_n$$

Similar to Pauli matrices, any two Pauli operators either commute or anti-commute.

What's meant by stabilization?

Given a quantum state $|\psi\rangle$, we say a unitary matrix U stabilizes $|\psi\rangle$ if $|\psi\rangle$ is an eigenvector of U with eigenvalue 1, ie

$$U|\psi\rangle = |\psi\rangle$$

If U and V both stabilize a quantum state $|\psi\rangle$, then UV, U^{-1} also stabilize $|\psi\rangle$

$$UV|\psi\rangle = U|\psi\rangle = |\psi\rangle$$

$$U^{-1}|\psi\rangle = U^{-1}U|\psi\rangle = |\psi\rangle$$

Instead of storing complex amplitudes, we now try to store the set formed by the stabilizer circuits for a quantum state $|\psi\rangle$, since this set is unique. Let's denote this as $Stab(|\psi\rangle)$. Normally writing $Stab(|\psi\rangle)$ requires 2^{2n} parameters.

We're interested in $Stab(|\psi\rangle) \cap P_n$

1. $|\psi\rangle$ is stabilized by exactly 2^n Pauli operators.
2. $|\psi\rangle$ is uniquely determined by $Stab(|\psi\rangle) \cap P_n$, or the group of Pauli operators that stabilize $|\psi\rangle$

Gottesman-Knill: State Representation

- Instead of 2^n state-vector amplitudes, track n generators $g_1, \ldots, g_n$
- For $|0\rangle^{\otimes n}$, the initial generators are:

$$g_1 = +ZII\ldots I, \quad g_2 = +IZI\ldots I, \quad \ldots, \quad g_n = +II\ldots IZ$$

- Each generator g_i stores:
  - $2n$ binary bits for Pauli operators (I, X, Y, Z)
  - 1 phase bit (0 for $+1$, 1 for -1)
- **Total memory:** $n(2n+1) = O(n^2)$ bits

Tableau Representation of Pauli Matrices

Each Pauli matrix on qubit j is encoded with two binary bits (x_j, z_j) and a sign bit $r \in \{0, 1\}$ for $(-1)^r$:

Pauli	x	z	Form	Action of H ($HPH^\dagger$)
I	0	0	I	$+I$ (phase unchanged)
X	1	0	X	$+Z$ (phase unchanged)
Z	0	1	Z	$+X$ (phase unchanged)
Y	1	1	iXZ	$-Y$ (**phase flips!**)

- **Memory:** $2n$ bits for Pauli matrices + 1 phase bit r per generator.

Gottesman-Knill: Unitary Gates

- If g_i stabilizes $|\psi\rangle$, then the stabilizer of $U|\psi\rangle$ is given by:

$$g_{i'} = U g_i U^\dagger$$

- Applying U transforms basis operators directly:
  - **Hadamard:** $X \leftrightarrow Z, \quad Y \to -Y$
  - **Phase:** $X \to Y, \quad Z \to Z$
  - **CNOT:** $X_1 \to X_1 X_2, \quad Z_2 \to Z_1 Z_2$
- Updating all n generators takes only $\boldsymbol{O(n)}$ **time** per gate

Hadamard Gate Update (H)

Applying Hadamard to qubit a executes two fast bitwise updates on every row i:

1. **Phase bit:** $r_i \leftarrow r_i \oplus (x_{ia} z_{ia})$
2. **Pauli bits:** Swap $x_{ia} \leftrightarrow z_{ia}$

Example: 1-Qubit Hadamard on $|0\rangle$

Row	Type	Initial ($\|0\rangle$)	Update Rule	Final ($\|+\rangle$)	New Op
R_1	Destabilizer	$+X$ `(1, 0)`	$r \leftarrow 0 \oplus (1 \cdot 0)$, swap	`(0, 1)`, $r = 0$	$+Z$
R_2	Stabilizer	$+Z$ `(0, 1)`	$r \leftarrow 0 \oplus (0 \cdot 1)$, swap	`(1, 0)`, $r = 0$	$+X$

Gottesman-Knill: Measurements

Measuring qubit a in the computational basis checks commutation with Z_a:

- **Case 1: Random Outcome (Anticommuting)**
  - Z_a anticommutes with at least one generator g_p
  - Multiply any other anticommuting g_j by g_p to make them commute
  - Replace g_p with $\pm Z_a$ (chosen randomly with 50% probability)
  - **Runtime:** $O(n^2)$
- **Case 2: Deterministic Outcome (Commuting)**
  - Z_a commutes with all g_i, meaning $\pm Z_a \in S$

- Must find $(c_1, ..., c_n) \in \{0, 1\}^n$ such that:

$$\pm Z_a = g_1^{c_1} g_2^{c_2} ... g_n^{c_n}$$

The Gottesman-Knill Bottleneck

- To determine whether the outcome is 0 or 1 in Case 2, we must solve:

$$\sum_{i=1}^{n} c_i \vec{g}_i = \vec{Z}_a \pmod 2$$

- Solving this linear system requires **Gaussian elimination over $\mathbb{F}_2$**
- **Runtime: $O(n^3)$** in practice
- For frequent measurements on large circuits ($n > 1000$), $O(n^3)$ becomes prohibitively slow

- **The Aaronson-Gottesman contribution:** Eliminate Gaussian elimination entirely using **destabilizers**, dropping measurement time to $O(n^2)$

Aaronson-Gottesman Fix: Destabilizers

The improvement is to store **twice as many generators**:

- $R_1, \ldots, R_n$: **destabilizers**
- $R_{n+1}, \ldots, R_{2n}$: stabilizers
- R_{2n+1}: one scratch row used during measurement

For the initial state $|0\rangle^{\otimes n}$:

$$R_i = X_i, \quad R_{n+i} = Z_i$$

The key invariant is:

$$R_i R_{n+i} = -R_{n+i} R_i$$

while R_i commutes with every other stabilizer generator.

So each destabilizer behaves like a **dual basis vector** for one stabilizer generator.

Memory: $2n(2n+1) \approx 4n^2$ bits.

Deterministic Measurement Without Gaussian Elimination

Suppose measuring qubit a is deterministic. Then:

$$\pm Z_a = \prod_{h=1}^{n} R_{n+h}^{c_h}$$

Originally we had to solve for the unknown coefficients c_h.

With destabilizers:

$$c_i = R_i \cdot Z_a$$

But R_i anticommutes with Z_a exactly when it contains X or Y on qubit a, i.e. when

$$x_{ia} = 1$$

Therefore:

$$\boxed{c_i = x_{ia}}$$

So we simply multiply the stabilizer rows R_{n+i} for every destabilizer with $x_{ia} = 1$.

The final sign tells us whether the measurement is 0 or 1.

Measurement Algorithm

Case	What happens?	Cost
Random	Z_a anticommutes with a stabilizer. Use that row to clear the other anticommuting rows and replace it by $\pm Z_a$.	$O(n^2)$
Deterministic	Use the destabilizer x_{ia} bits to directly identify which stabilizers multiply to $\pm Z_a$.	$O(n^2)$

The helper `rowsum(h,i)` performs

$$R_h \leftarrow R_i R_h$$

by XORing the x, z bits and separately updating the phase bit.

This is the central improvement of the paper.

Cost of the Improved Tableau Simulator

Operation	Direct Gottesman-Knill	Aaronson-Gottesman
Hadamard / Phase / CNOT	$O(n)$	$O(n)$
Random measurement	$O(n^2)$	$O(n^2)$
Deterministic measurement	about $O(n^3)$	$\boldsymbol{O(n^2)}$
Memory	$n(2n+1)$ bits	$2n(2n+1)$ bits

CHP: Practical Implementation

The authors implemented the improved algorithm in a C simulator called **CHP** — CNOT-Hadamard-Phase.

Instruction	Operation
`c a b`	CNOT $a \to b$
`h a`	Hadamard on a
`p a`	Phase on a
`m a`	Measure a

The aim was high performance for **thousands of qubits** and circuits containing frequent measurements.

CHP Experiment: Why Some Measurements Are Slower

The authors generated random circuits containing

$$\lfloor \beta n \log_2 n \rfloor$$

unitary gates and then measured every qubit.

They observed:

- $\beta = 0.6$: measurement time grew roughly linearly with n.
- $\beta = 1.2$: measurement time grew roughly quadratically with n.

- Intermediate β values lay between the two.

Reason: random gates gradually make the tableau **dense**.

A measurement performs `rowsum` for rows with $x_{ia} = 1$:

$$\text{more 1s in tableau} \rightarrow \text{more row additions} \rightarrow \text{slower measurement}$$

The paper describes this as a phase-transition-like change near $\Theta(n \log n)$ random gates.

Computational Complexity: $\oplus\, L$

Gottesman-Knill simulation problem is complete for

$$\boxed{\oplus\,\mathrm{L}}$$

("parity-L").

A useful intuition for $\oplus\, L$ is:

- problems reducible to polynomial-size NOT/CNOT circuits,
- essentially binary linear/parity computation over $\mathbb{F}_2$.

Thus Hadamard and phase gates do not make stabilizer circuits computationally as powerful as general classical circuits.

Canonical Form.

Any unitary stabilizer circuit has an equivalent circuit with only 11 structured rounds:

$$\boxed{\mathrm{H}\text{—}\mathrm{C}\text{—}\mathrm{P}\text{—}\mathrm{C}\text{—}\mathrm{P}\text{—}\mathrm{C}\text{—}\mathrm{H}\text{—}\mathrm{P}\text{—}\mathrm{C}\text{—}\mathrm{P}\text{—}\mathrm{C}}$$

Here each letter represents an entire layer containing only that gate type.

The proof works on the $2n \times 2n$ tableau

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix}$$

and repeatedly uses:

- Hadamards to make a block full rank,
- CNOTs for binary Gaussian elimination,
- phase gates to modify symmetric blocks.

Eventually the tableau is reduced to

$$\begin{pmatrix} I & 0 \\ 0 & I \end{pmatrix}$$

Gate Count from the Canonical Form

A known result shows that any CNOT circuit can be synthesized using

$$O\left(\frac{n^2}{\log} n\right)$$

CNOT gates.

The canonical form contains only a constant number of CNOT blocks, and the Hadamard/phase blocks need only $O(n)$ gates.

Therefore:

Any unitary stabilizer circuit has an equivalent circuit with $O\left(\frac{n^2}{\log} n\right)$ gates

The paper also notes a construction with

$$O(\log n)$$

parallel depth using $O(n^2)$ gates.

Open Problems

The paper ends with several directions:

- Can deterministic measurements be reduced from $O(n^2)$ to $O(n)$?
- Can the 11-round canonical form be shortened?
- Can local rewrite rules efficiently optimize arbitrary stabilizer circuits?
- Is there a gate set between “classically easy” and “universal quantum”?
- What happens with arbitrary product inputs if all measurements are delayed?

- Can we efficiently synthesize an approximately minimum-size Clifford circuit?

These questions explore the boundary between classical simulation, circuit optimization, and universal quantum computation.

Main Takeaways

Idea	Result
State representation	$O(n^2)$ bits instead of 2^n amplitudes
Destabilizers	Remove Gaussian elimination during measurement
Measurement	$O(n^2)$ instead of about $O(n^3)$
Implementation	CHP makes thousands of qubits practical
Complexity	$\oplus L$-complete
Circuit synthesis	11-round canonical form and $O\left(\frac{n^2}{\log} n\right)$ gate bound
Extensions	Mixed states + limited non-stabilizer resources

The central message is:

Store how Pauli operators transform, not the full quantum state.

Reference

Scott Aaronson and Daniel Gottesman,
Improved Simulation of Stabilizer Circuits,
arXiv:quant-ph/0406196, version 5 (2008).